

Tic Tac Toe (tkinter GUI) — Brief

Project explainer document

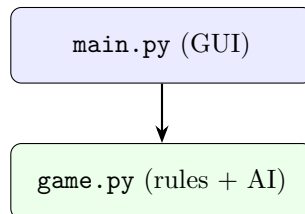
1 What it is

A tkinter-based Tic Tac Toe game with two modes: hot-seat two-player, and single-player against a minimax-driven AI opponent that is provably unbeatable (it always wins or draws, never loses).

2 Architecture

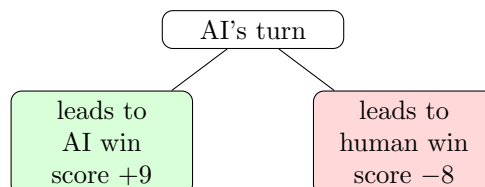
Three pieces:

- `game.py` – board rules, win/draw detection, and the minimax AI. No tkinter dependency, so it's independently unit-testable.
- `main.py` – the tkinter window: the 3x3 button grid, a mode toggle (two player / vs AI), a mark-choice control, status and score labels, and the logic that schedules the AI's move after the human's turn.
- `tests/` – unit tests for the AI's decisions plus a script that plays out hundreds of full games to prove it never loses.



3 Minimax, in brief

Minimax explores the entire tree of possible future moves (the **game tree**) from the current board, scores every way the game could end (win for the AI: positive; win for the human: negative; draw: zero), and picks, at each of the AI's turns, whichever move leads to the best score *assuming the human always replies with their own best move*. That last assumption – planning against the opponent's best play, not their average or worst play – is what makes the result airtight rather than merely "usually good."



The AI, maximizing, takes the left branch. On the human's simulated turns the same search assumes the human minimizes the AI's score, i.e. always finds their best reply. Because Tic Tac Toe's game tree is small enough to search completely, this isn't a heuristic; it's a full, correct evaluation of every possible outcome. **Alpha-beta pruning** (tracking the best-guaranteed scores for each side and cutting off branches that can no longer change the final decision) is layered on top purely for speed; it changes nothing about the result, and on a board this small the AI decides its move in well under a tenth of a second even without it.

4 Turn flow in Vs AI mode

Human clicks a cell → move applied and drawn → if the game isn't over, the AI's move is computed instantly but applied after a deliberate 400 millisecond delay (via `root.after`, so the window stays responsive) → if the human plays second, the AI's opening move is scheduled the same way as soon as a new round starts.

5 Verification: is it actually unbeatable?

Three checks, each run for both starting orders (human first, AI first):

1. **Exhaustive search** over every legal human move at every human turn (a complete enumeration, not a sample): 569 distinct games when the human moves first, 73 when the AI moves first. **AI losses: 0** in both.
2. **1000 random-play games** per starting order (human plays uniformly random legal moves). **AI losses: 0** in both (789/211 win/draw human-first; 996/4 win/draw AI-first).
3. **Optimal vs. optimal** (both sides use minimax): draws in both starting orders, exactly as game theory predicts.

Reproducible with `python3 tests/verify_unbeatable.py`. Six additional unit tests in `tests/test_ai.py` check specific tactical situations (taking an immediate win, blocking an immediate threat, preferring a win over a block when both exist) directly, all passing.

6 What's unchanged

Two-player mode's rules, win detection, and score tracking are exactly as before. The AI mode adds a second, independent score dictionary (human/AI/draws) rather than altering the existing one.